

MODULE-14

CONTAINERIZATION USING DOCKER

Docker

Docker is a tool that simplifies the process of developing, packaging, and deploying applications.

Docker is an OS-level virtualization (or containerization) platform, which allows applications to share the host OS kernel instead of running a separate guest OS like in traditional virtualization.

- **Portability:** Runs anywhere in local machine, cloud, on-prem servers.
- **Consistency:** Same behaviour in development, testing, and production.
- **Lightweight:** No full OS per app; containers share the host kernel.
- **Scalability:** Ideal for microservices and orchestrators like Kubernetes and Docker Swarm.
- **Efficiency:** Starts in seconds, uses fewer system resources.

Basic Docker Commands:

1. **docker run:** Creates and starts a new container from an image.
2. **docker ps:** Lists running container
3. **docker stop:** Stops a running container
4. **docker rm:** Removes one or more stopped containers.
5. **docker images:** lists all Docker images stored on the local system.
6. **docker rmi <image-id>:** Removes a Docker image
7. **docker pull:** Downloads a Docker image from a registry such as Docker Hub.
8. **docker exec:** runs a command inside an already running container.
9. **Append Command:**
 - add a command after the image name to execute I when the container starts

Docker Container

It is a lightweight, standalone, executable software package. It includes to run a piece of software:

- application code
- runtime

- system tools
- system libraries
- settings

Docker Run Commands

1. `docker run`: creates and stores a new container from a Docker image.
 - Syntax: `docker run [OPTIONS] IMAGE [COMMAND]`
2. Run Container with a Specific Name:
 - use the `--name` option to assign a custom name
 - `docker run --name mycontainer nginx`
3. Run a Container in Detached Mode:
 - Use the `-d` option to run the container in the background
 - `docker run -d nginx`
4. Run a Container Interactively:
 - Use `-it` to interact with container through the terminal
 - `docker run -it ubuntu bash`
5. Run a Container and publish ports:
 - use `-p` to host port to a container port.
 - `docker run -p <host-port>:<container-port> IMAGE`
 - `docker run -p 8080:80 nginx`
6. Run a Container and remove it after exit:
 - use `--rm` option to automatically delete the container when it stops
 - `docker run --rm ubuntu echo "Hello"`

Docker File

Dockerfile is a plain-text that lists ordered instructions the Docker builder runs to assemble an image and automate creating containers.

Each instruction appears on its own line with any arguments after it, and the builder executes lines in order to produce a layered, reproducible image.

- Instruction keywords (e.g., `FROM`, `RUN`) come first and are usually written in uppercase.
- Arguments are written on the same line as the instruction.
- Comments start with `#` and are ignored during the build.
- Each instruction creates a new image layer, affecting image size and build cache.
- Use a `.dockerignore` file to exclude unnecessary files from the build context and speed up builds.

Docker Instructions:

Docker Instructions are the commands written inside a Dockerfile that tell Docker how to build a Docker image.

1. **FROM:** defines which image to download and start from.
 - must be the first command in your Dockerfile
 - can have multiple FROM statements i.e. Dockerfile produces more than 1 image.
2. **MAINTAINER:** documentation that defines the author who author is or who to contact if it has bugs.
3. **RUN:** executes commands during Docker image build process and saves the result in a new image layer
 - mainly used to install packages, create directories, or configure the environment.
4. **ADD:** gives instructions to copy new files, directories, or remote file URLs and then adds them to the filesystem of the image.
 - can also extract archives or download files from URLs.
5. **ENV:** ENV statement sets the environment variables both during the build and when running the result.
6. **ENTRYPOINT:** specifies the starting of the expression to us when starting your container.
 - Simply ENTRYPOINT specifies the start of the command to run.
7. **CMD:** Specifies the default command to run when the container starts.
8. **EXPOSE:** Specifies the port the application uses.
9. **LABEL:** Adds metadata (e.g., author, version) to the image.

Docker Image

Docker Image is a lightweight, standalone and executable software package that includes everything needed to run an application.

It contains everything needed to run an application, including:

- Application code
- Runtime
- Libraries and dependencies
- System tools and configuration files

Docker image are OS-specific and CPU-architecture specific(A Docker image is built to run on a specific operating system and CPU architecture).

A single image can be used to create multiple identical containers.

Components of a Docker Image

A Docker image isn't one large, monolithic file. It's cleverly constructed from a series of layers and stored in a registry.

1. **Layers:** An image is composed of a stack of read-only layers.
 - Each instruction in a Dockerfile (like FROM, COPY, RUN) creates a new layer on top of the previous one.
2. **Base Image:** This is the foundational layer of your image, specified by the FROM instruction in a Dockerfile.
3. **Docker Registries:** A Docker Registry is a storage and distribution system for Docker images.

Important:

1. **Image Layers:**
 - A Docker image is made up of multiple read-only layers.
 - Each Dockerfile instruction (e.g., RUN, COPY) creates a new layer, enabling efficient storage and caching.
2. **Container Layer:**
 - When a container starts, Docker adds a writable layer on top of the image layers.
 - All changes made by the running container are stored in this layer.
3. **Parent Image:**
 - A parent image is an existing image used as the starting point for creating a new image.
 - It is specified using the FROM instruction in a Dockerfile.
4. **Base Image:**
 - A base image is the original image with no parent image.
 - Examples: scratch, ubuntu, alpine.
5. **Docker Manifest:**
 - A Docker manifest is a metadata file that describes a Docker image.
 - It contains information such as image layers, tags, architecture, and operating system.
6. **Container Registries:**
 - A container repository is a collection of related Docker images stored inside a registry.
 - It may contain multiple image versions identified by tags.

Creating a Docker Image – Interactive Method

- Create and start a container from an existing image using ***docker run***.
- Make the required changes manually inside the running container (e.g., install software, modify files).
- Save the modified container as a new image using ***docker commit***.

Creating a Docker Image – Dockerfile Method

- Create a Dockerfile containing the instructions to build the image.
- Specify the base image, copy application files, install dependencies, and define the startup command.
- Build the image using the docker build command.

Docker Compose

Docker compose is a tool used to define and manage multi-container Docker applications.

It uses docker-compose.yml to configure services.

- Manages multiple containers with a single command.
- Simplifies application deployment and configuration.
- Ensures consistent environments across systems.
- Automatically creates networks and volumes

Basic Docker Commands

1. **docker compose up** – Create and start containers.
2. **docker compose down** – Stop and remove containers, networks, and volumes.
3. **docker compose start** – Start existing containers.
4. **docker compose stop** – Stop running containers.
5. **docker compose ps** – List Compose services.
6. **docker compose logs** – View container logs.

Install Docker Compose

1. Update the package manager
2. Download the software
3. Apply the permissions
4. Verify the download software

Docker Engine

- Docker Engine is the core software that builds, runs and manages Docker containers.
- It consists of the Docker CLI, Docker Daemon and REST API.

Docker CLI

- A command-line interface used to interact with Docker.
- eg: docker run, docker build

Docker REST API

- Allows the application to communicate with the Docker Daemon programmatically

Docker Daemon

- A background service (dockerd) that builds, runs and manages Docker images, containers, networks, volumes.

Docker Storage

1. Storage Drivers:

- Manage how image and container layers are stored.
- Improve storage efficiency using layered file systems.

2. Data Volumes:

- Persistent storage that exists independently of containers.
- Used to store data permanently and share data between containers

Changing the Storage Driver

- Configure a different storage driver in Docker settings or the Docker daemon configuration.
- Requires restarting the Docker service.

Creating a Volume

- Use ***docker volume create <volume_name>*** to create a new volume.

Listing All Volumes

- Use ***docker volume ls*** to display all available volumes.

Docker Networking

Docker Networking refers to the set of mechanisms and technologies Docker provides for communication between Docker containers, as well as between containers and the outside world.

Default Networks

1. **Bridge:** Default network for containers on a single host.
2. **Host:** Shares the host's network stack.
3. **None:** Disables networking for the container.

Listing all Docker Networks:

- Use `docker network ls` to list all Docker networks.

Inspecting a Docker Network:

- Use `docker network inspect <network_name>` to view network details.

Creating a New Network:

- Use `docker network create <network_name>` to create a custom network.
- Enables secure communication between selected containers.

Container Orchestration

The automated management, deployment, scaling, networking, and monitoring of containers.

Need for Container Orchestration:

- Manages large numbers of containers efficiently.
- Automates deployment, scaling, and recovery from failures.
- Ensures high availability and load balancing.

Benefits:

- Automated deployment and scaling.
- High availability and fault tolerance.
- Load balancing between containers.
- Efficient resource utilization.
- Easier management of containerized applications.

Kubernetes

- Kubernetes (K8s) is an open-source container orchestration platform.
- It automates deployment, scaling, networking, and management of containerized applications.